A1. (Regular Expressions)

(a) $\Sigma = \{0,1\}$ üzerinde, en az 0 içeren stringler

$$(0+1)^* 0 (0+1)^*$$

Açıklama: İstediğimiz yerde 0 olabilir, öncesi ve sonrası herhangi bir 0/1 dizisi

Kabul: 0, 10, 01, 1001, 1100

Ret: ε, 1, 11, 1111

(b) $\Sigma = \{a,b\}$ üzerinde, "aa" alt dizisini içermeyen stringler.

$$b^* (ab^*)^*$$

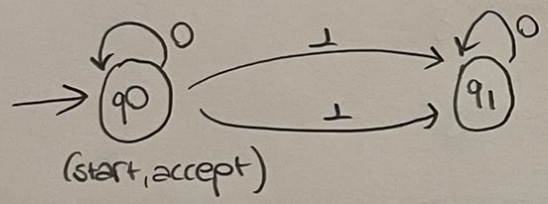
Açıklama: Her a'dan sonra sadece b'ler gelebilir, başka bir 'a' gelmeden önce en az bir b olmalı

Kabul: ε, b, ab, abb, bab, bbab

Ret: a, aa, aab, baaa

A2. (Finite Automata)

$\Sigma = (0,1)$ üzerinde, toplamda çift sayıda 1 içeren stringleri kabul eden DFA



Geçiş tablosu

δ	0	1
→*q0	q0	q1
q1	q1	q0

Açıklama

→ q0: görülen 1 sayısı çift (başlangıç ve kabul durumu)

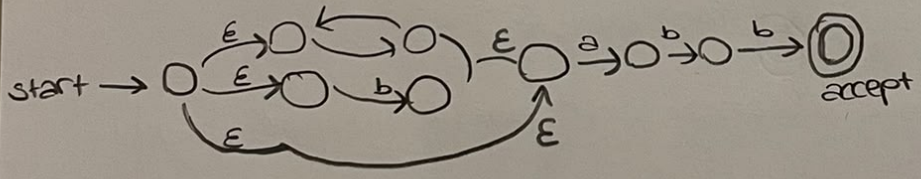
→ q1: görülen 1 sayısı tek

→ 0 okununca aynı

→ 1 okununca diğer durum

A3. (Thompson's Construction)

$R = (a+b)^* abb$ ifadesi için ε-NFA



Açıklama

→ (a+b) için iki kollu yapı

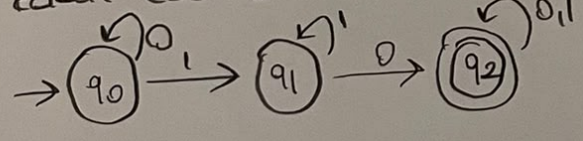
→ * için başlangıca geri dönen ε geçişleri

→ sonra "abb" için ardışık geçişler eklendi

→ son durum kabul durumu

A4. (DFA and Minimization)

$L = \{w \in \{0,1\}^* \mid w$, "01" alt dizisini içerir$\}$ dilini kabul eden DFA.



δ	0	1
→q0	q0	q1
q1	q2	q1
*q2	q2	q1

Açıklama

→ q0: hiçbir "0" görülmedi veya son sembol 0 değil

→ q1: son sembol 0 (01 olabilir)

→ q2: kabul durumu

→ q2'den sonra tüm stringler kabul edilir.

Minimizasyon

Bu DFA zaten minimaldir. Bu durumun hepsi farklı davranışlar gösterir

Ceren Simmek
21120205041O

A5. (Short Answers)

a) Regular ve context-free temel fark?

→ Regular diller sonlu otomatlar ile tanımlanabilir ve yalnızca düzenli yapıları ifade edebilir.

Context-free diller ise yığınlı otomatlar ile tanımlanır ve iç içe yapılar gibi daha karmaşık yapıları ifade edebilir. Her regular dil bir context-free dildir ancak her context-free dil regular değildir.

b) Regular dillerin pratikte iki kullanım alanı?

1) Derleyicilerde lexical analysis (token tanıma)

2) Metin işleme araçlarında pattern matching.

c) ε-geçişlerinin ε-NFA'lara sağladığı avantaj?

→ ε-geçişleri, herhangibir girdi sembolü tüketmeden durumlar arasında geçiş yapmayı sağlar. Bu sayede düzenli ifadelerin NFA'lara daha basit ve sistematik bir şekilde dönüştürülmesi mümkün olur.

d) Bir DFA'nın minimal olduğunu nasıl anlarsınız?

→ DFA'nın durumları eşdeğerlik sınıflarına ayrılır. Eşdeğer durumlar birleştirildikten sonra kalan DFA'da daha fazla durum birleştirilemiyorsa DFA minimaldir.

Ceren Sinmez
21120205010